

1. **State and rendering:** State, props and derived values · useState and what a re-render is · State is a snapshot of one render · Updater functions: new state from the previous state · Batching: many updates, one render · Don't store what you can compute
2. **Updating objects and arrays:** Updating objects in state: replace, never mutate · Updating arrays: spread, map and filter
3. **Effects:** Effects: synchronising with something outside React · The dependency array: none, [] and [x] · State closures in effects and timers · Cleanup: before the next effect and on unmount · StrictMode: why effects run twice in development · You might not need an effect
4. **Loading data:** Fetching data in an effect · Loading, error, empty and success · Cancelling a fetch with AbortController
5. **Hooks: rules and reuse:** The rules of hooks (and why) · Custom hooks: reusing stateful logic · A custom hook for loading data: useFetch · A custom hook for persistence: useLocalStorage

State and rendering

State, props and derived values

An interface shows three kinds of data: **props** (given by the parent, read-only), **state** (remembered by the component and changed over time, usually by the user) and **derived values** (anything you can compute from the other two during render).

Which kind of data is it?

Value	Kind	Why
the list of tasks passed by the parent	prop	someone else owns it
the text typed in the search box	state	changes over time, nobody else knows it
the tasks that match the search	derived	computed from the list and the text
"3 of 5 done"	derived	counted from the list
whether a "Show done" checkbox is ticked	state	the user changes it

```
import { useState } from 'react';

const TASKS = [
  { id: 1, title: 'Write the API skeleton', done: true },
  { id: 2, title: 'Add full CRUD', done: false },
  { id: 3, title: 'Document the endpoints', done: true },
];

function TaskSummary({ tasks }) { // prop
  const [showDone, setShowDone] = useState(true); // state
  const visible = showDone ? tasks : tasks.filter((t) => !t.done); // derived
  const doneCount = tasks.filter((t) => t.done).length; // derived

  return (
    <section>
      <label>
        <input type="checkbox" checked={showDone}
          onChange={(e) => setShowDone(e.target.checked)} /> Show done tasks
      </label>
      <p>{doneCount} of {tasks.length} done</p>
      <ul>
        {visible.map((t) => <li key={t.id}>{t.done ? '✓ ' : ''}{t.title}</li>)}
      </ul>
    </section>
  );
}

function App() {
  return <TaskSummary tasks={TASKS} />;
}
```

Example. The box below receives tasks as a prop, keeps one piece of state (showDone) and derives two values (visible and doneCount). Untick the checkbox: only the state changes, and React recomputes both derived values on the next render.

Common mistake. Copying a prop into state: `const [tasks, setTasks] = useState(props.tasks)`. `useState` reads its argument **only on the first render**, so when the parent later passes a new list, this component keeps showing the old one. Read the prop directly; make a copy in state only when you deliberately want an editable draft that ignores later changes.

useState and what a re-render is

`const [value, setValue] = useState(initial)` **gives a component one piece of memory**. Calling `setValue(next)` **stores the new value and asks React to re-render: to call the component function again and update the page with what it returns**.

- Render = React calls your component. Commit = React updates the DOM with the difference.
- A setter called with the **same** value (`Object.is`) skips the render.
- Each component instance has its own state: two `<Counter />` elements count separately.

```
import { useState } from 'react';

function App() {
  const [count, setCount] = useState(0);
  let clicks = 0; // a plain local variable
  console.log(`App rendered: count = ${count}, clicks = ${clicks}`);

  return (
    <>
      <button onClick={() => { clicks++; console.log('clicks is now', clicks); }}>
        Plain variable: {clicks}
      </button>
      <button onClick={() => setCount(count + 1)}>
        State: {count}
      </button>
    </>
  );
}
```

Example. Click both buttons below and watch the box's console. The `let clicks` variable is increased, but nothing re-renders and the next render starts it from 0 again. The state button re-renders, and the log shows the component running again with the new value.

Common mistake. Assigning to the state variable: `count = count + 1`. With `const` it throws a `TypeError`; with `let` it runs, but nothing tells React, so the screen never changes and the next render brings the old value back. Always go through the setter: the variable is a read-only copy for this render.

State is a snapshot of one render

Inside one render, a state variable is a constant. Calling the setter does not change it; it queues a new value for the next render. That is why `setCount(count + 1)` twice in one click adds 1, not 2.

```
import { useState } from 'react';

function App() {
  const [count, setCount] = useState(0);

  function handleClick() {
    setCount(count + 1);
    setCount(count + 1);
    setCount(count + 1);
    console.log('right after three setters, count is', count);
    setTimeout(() => console.log('3 s later this click still sees', count), 3000);
  }
}
```

```
return <button onClick={handleClick}>+3? Count: {count}</button>;
}
```

Example. Click **+3?** below: it calls `setCount(count + 1)` three times and logs `count` afterwards. The console says 0, and the screen shows 1. Then use the render-cycle visualiser under this card: choose "Two `setCount(count + 1)` calls" and step through the queue.

Common mistake. Reading state right after setting it to decide something: `setQuery(text); fetchResults(query);` sends the **old** query. Use the value you just computed (`fetchResults(text)`), or let an effect react to the new query after the render.

Updater functions: new state from the previous state

Pass the setter a function instead of a value, `setCount((c) => c + 1)`, and React calls it with the **latest** queued value. Use it whenever the next state depends on the previous one.

```
setCount(count + 1);           // "make it 1" (if count is 0 in this render)
setCount((c) => c + 1);        // "add one to whatever it is by then"

// Mixing: value then updater
setCount(10);                 // queue: replace with 10
setCount((c) => c * 2);        // queue: 10 → 20 (next render: 20)
```

```
import { useState } from 'react';

function App() {
  const [count, setCount] = useState(0);

  function addThree() {
    setCount((c) => c + 1);
    setCount((c) => c + 1);
    setCount((c) => c + 1);
  }

  function resetThenOne() {
    setCount(0);               // a value: replace
    setCount((c) => c + 1);    // an updater: receives 0
  }

  return (
    <>
      <p>Count: {count}</p>
      <button onClick={addThree}>+3</button>
      <button onClick={resetThenOne}>Reset then +1</button>
    </>
  );
}
```

Example. The box has the same three-call button, this time with updaters, so a click adds 3. The **Reset then +1** button queues a value and then an updater: the queue goes "replace with 0" → "0 + 1", and the screen shows 1.

Common mistake. Using an updater everywhere "to be safe" while still reading the stale variable inside it: `setTotal((t) => t + price * count)` is fine only if `price` and `count` are current. The updater protects the variable it receives, not every other variable it closes over.

Batching: many updates, one render

React waits until the code that queued updates has finished (an event handler, a timer callback, a promise callback) and then renders **once** for all of them. This is called **batching**.

```
import { useState } from 'react';

function App() {
  const [count, setCount] = useState(0);
```

```
const [status, setStatus] = useState('idle');
console.log(`render: count = ${count}, status = ${status}`);

function handleClick() {
  setCount((c) => c + 1);
  setStatus('saving');           // same batch as the line above
  setTimeout(() => {
    setCount((c) => c + 1);
    setStatus('saved');         // batched too, inside the timer
  }, 1000);
}

return <button onClick={handleClick}>{status}: {count}</button>;
}
```

Example. Click the button below. The handler updates two variables and also waits for a timer before updating them again. The console shows one render per batch: two renders for the whole click, not four.

Common mistake. Expecting a render (and a log in the component body) per setter call, and splitting related state into many variables "so they update separately". They do not: every update of the same event lands in the same render. If two values always change together, consider one state object or one derived value.

Don't store what you can compute

If a value can be calculated from props or existing state, compute it **during render** instead of keeping it in state. Duplicated state has to be kept in sync by hand, and it eventually goes out of sync.

```
// Duplicated state kept in sync with an effect: two renders, one of them wrong
const [tasks, setTasks] = useState([]);
const [visible, setVisible] = useState([]);
useEffect(() => {
  setVisible(tasks.filter((t) => t.title.includes(search)));
}, [tasks, search]);

// Derived during render: one render, always right
const visible = tasks.filter((t) => t.title.toLowerCase().includes(search.toLowerCase()));
```

```
import { useState } from 'react';

const START = [
  { id: 1, title: 'Write the API skeleton', done: true },
  { id: 2, title: 'Add full CRUD', done: false },
  { id: 3, title: 'Return the right status codes', done: false },
];

function App() {
  const [tasks, setTasks] = useState(START);
  const [search, setSearch] = useState('');

  const visible = tasks.filter((t) => t.title.toLowerCase().includes(search.toLowerCase()));
  const pending = tasks.filter((t) => !t.done).length;

  const toggle = (id) => setTasks(tasks.map((t) => (t.id === id ? { ...t, done: !t.done } : t)));

  return (
    <>
      <input placeholder="Search" value={search} onChange={(e) => setSearch(e.target.value)} />
      <p>{pending} pending · showing {visible.length}</p>
      <ul>
        {visible.map((t) => (
          <li key={t.id}>
            <label><input type="checkbox" checked={t.done} onChange={() => toggle(t.id)} /> {t.title}</label>
          </li>
        ))}
      </ul>
    </>
  );
}
```

```
);
}
```

Example. The box filters a task list as you type. There are exactly two pieces of state, tasks and search; visible and the counter are computed in the body. Toggle a task and the counter stays correct with no extra code.

Common mistake. Storing the selected object: `const [selected, setSelected] = useState(task)`. Rename the task in the list and the detail panel still shows the old title, because it holds a separate copy. Store `selectedId` and derive `const selected = tasks.find((t) => t.id === selectedId)`.

Updating objects and arrays

Updating objects in state: replace, never mutate

Treat objects in state as read-only. To change one field, create a new object with the spread syntax, `setTask({...task, done: true})`, and pass it to the setter. Changing the existing object (`task.done = true`) does not re-render.

```
const [task, setTask] = useState({ id: 1, title: 'Add full CRUD', done: false });

// Wrong: same object, React sees no change
task.done = true;
setTask(task);

// Right: a new object
setTask({ ...task, done: true });

// Nested: copy each level you touch
setUser({ ...user, address: { ...user.address, city: 'Madrid' } });
```

```
import { useState } from 'react';

function App() {
  const [task, setTask] = useState({ title: 'Add full CRUD', done: false });

  function mutate() {
    task.done = !task.done; // edits the object of this render
    setTask(task);           // same reference: React skips the render
  }

  function replace() {
    setTask({ ...task, done: !task.done });
  }

  return (
    <>
      <input value={task.title} onChange={(e) => setTask({ ...task, title: e.target.value })} />
      <p>{task.title}: {task.done ? 'done' : 'pending'}</p>
      <button onClick={mutate}>Toggle (mutating)</button>
      <button onClick={replace}>Toggle (new object)</button>
    </>
  );
}
```

Example. The box has a small edit form whose state is one object. **Toggle (mutating)** changes the object in place and calls the setter: nothing happens on screen. **Toggle (new object)** works. Type in the title after a mutating click and the hidden change suddenly appears, because typing causes a render.

Common mistake. Believing `const` protects the object. `const [task, setTask]` only stops you from reassigning the name; `task.done = true` is still allowed and is exactly the mutation React cannot see.

Updating arrays: spread, map and filter

Add with spread (`[...tasks, newTask]`), **remove with filter**, **change one item with map** and **a new object, and sort a copy** (`[...tasks].sort()` or `toSorted()`). **Never push, splice, assign by index or sort the array in state.**

Array operations on state

Goal	Avoid (mutates)	Use (new array)
add	<code>tasks.push(t)</code>	<code>[...tasks, t]</code>
remove	<code>tasks.splice(i, 1)</code>	<code>tasks.filter((x) => x.id !== id)</code>
change one	<code>tasks[i].done = true</code>	<code>tasks.map((x) => x.id === id ? { ...x, done: true } : x)</code>
sort	<code>tasks.sort(cmp)</code>	<code>[...tasks].sort(cmp)</code> OR <code>tasks.toSorted(cmp)</code>
insert at i	<code>tasks.splice(i, 0, t)</code>	<code>[...tasks.slice(0, i), t, ...tasks.slice(i)]</code>

```
import { useState } from 'react';

let nextId = 3;

function App() {
  const [tasks, setTasks] = useState([
    { id: 1, title: 'Write the API skeleton', done: true },
    { id: 2, title: 'Add full CRUD', done: false },
  ]);

  const add = () => setTasks([...tasks, { id: nextId++, title: `Task ${nextId - 1}`, done: false }]);
  const addWithPush = () => {
    tasks.push({ id: nextId++, title: 'Pushed task', done: false });
    console.log('array length is now', tasks.length);
    setTasks(tasks); // same array: no render
  };

  const toggle = (id) => setTasks(tasks.map((t) => (t.id === id ? { ...t, done: !t.done } : t)));
  const remove = (id) => setTasks(tasks.filter((t) => t.id !== id));

  return (
    <>
      <button onClick={add}>Add</button> <button onClick={addWithPush}>Add with push</button>
      <ul>
        {tasks.map((t) => (
          <li key={t.id}>
            <label><input type="checkbox" checked={t.done} onChange={() => toggle(t.id)} /> {t.title}</label>
            <button onClick={() => remove(t.id)} aria-label={`Delete ${t.title}`}>X</button>
          </li>
        ))}
      </ul>
    </>
  );
}
```

Example. The box is a complete little task list with add, toggle and delete, all with new arrays. The **Add with push** button shows the broken version: the console proves the item went into the array, but the screen does not show it until some other update renders.

Common mistake. Sorting for display with `tasks.sort(...)` inside the JSX or the body. It reorders the state array in place during render, so the "original" order is lost and other components that share the array see it change. Sort a copy, or better, derive a sorted copy: `const sorted = tasks.toSorted(byTitle)`.

Effects

Effects: synchronising with something outside React

An effect (`useEffect(setup, deps)`) is code that runs **after** React has updated the page, to keep something outside React in sync with the current props and state: a timer, a network request, a browser API, an event

listener, a connection.

```
import { useEffect } from 'react';

useEffect(() => {
  // setup: runs after the commit
  const id = setInterval(tick, 1000);
  return () => clearInterval(id);    // cleanup: undoes the setup
}, []);                             // dependencies: when to re-synchronise
```

```
import { useState, useEffect } from 'react';

function App() {
  const [tracking, setTracking] = useState(false);
  const [pos, setPos] = useState({ x: 0, y: 0 });

  useEffect(() => {
    if (!tracking) return;          // nothing to synchronise
    console.log('subscribe to pointermove');
    const onMove = (e) => setPos({ x: e.clientX, y: e.clientY });
    window.addEventListener('pointermove', onMove);
    return () => {
      console.log('unsubscribe');
      window.removeEventListener('pointermove', onMove);
    };
  }, [tracking]);

  return (
    <div style={{ minHeight: 160 }}>
      <label>
        <input type="checkbox" checked={tracking} onChange={(e) => setTracking(e.target.checked)} />
        Track the pointer
      </label>
      <p>x = {pos.x}, y = {pos.y}</p>
    </div>
  );
}
```

Example. The box subscribes to the pointer position only while the checkbox is ticked. The effect depends on tracking: ticking starts a pointermove listener, unticking runs the cleanup, which removes it. Move the pointer over the preview and watch the console.

Common mistake. Calling fetch, setInterval or addEventListener directly in the component body. The body runs on **every** render: each keystroke adds another listener or timer, and a setState in the body triggers another render, which runs the body again: an infinite loop.

The dependency array: none, [] and [x]

The second argument of `useEffect` lists the values the effect reads. **No array:** run after every render. **[]:** run after the first render only. **[a, b]:** run after the first render and again whenever a or b changed since the last run (compared with `Object.is`).

When does the effect run?

Second argument	Runs after	Typical use
(none)	every render	rare: syncing with something that depends on everything
[]	the first render only	a connection or listener that never changes
[roomId]	the first render and when roomId changes	fetch or subscribe for the current id

```
import { useState, useEffect } from 'react';

function App() {
  const [count, setCount] = useState(0);
  const [text, setText] = useState('');
  console.log(`render: count = ${count}, text = "${text}"`);
}
```

```

useEffect(() => {
  console.log('A (no array): after every render');
});
useEffect(() => {
  console.log('B ([]): after the first render only');
}, []);
useEffect(() => {
  console.log(`C ([count]): count is ${count}`);
}, [count]);

return (
  <>
    <button onClick={() => setCount(count + 1)}>+1</button>
    <input value={text} onChange={(e) => setText(e.target.value)} />
  </>
);
}

```

Example. The box has three effects, one of each kind, and logs when each runs. Click +1 and type in the input: after the click A and C run; after typing only A runs. The render-cycle visualiser has the same component as the scenario "No array, [] and [count]".

Common mistake. Removing a dependency to stop an effect from running "too often", for example deleting query from [query]. The effect stops re-running, but it now fetches the old query forever. If an effect runs too often, fix the reason (an object created in render, a value that should be derived), never the array.

Stale closures in effects and timers

A function created during a render **closes over** that render's values. If an effect keeps such a function alive (an interval, a listener) without re-running, the function keeps reading old values: a **stale closure**.

```

import { useState, useEffect } from 'react';

function StaleTicker({ count }) {
  useEffect(() => {
    const id = setInterval(() => console.log(`stale ticker sees ${count}`), 1000);
    return () => clearInterval(id);
  }, []); // count is missing: the linter would warn
  return <p>Stale ticker running</p>;
}

function FixedTicker({ count }) {
  useEffect(() => {
    const id = setInterval(() => console.log(`fixed ticker sees ${count}`), 1000);
    return () => clearInterval(id);
  }, [count]); // a new interval for every count
  return <p>Fixed ticker running</p>;
}

function App() {
  const [count, setCount] = useState(0);
  const [fixed, setFixed] = useState(false);
  return (
    <>
      <button onClick={() => setCount(count + 1)}>Count: {count}</button>
      <label>
        <input type="checkbox" checked={fixed} onChange={(e) => setFixed(e.target.checked)} />
        Use the fixed ticker
      </label>
      {fixed ? <FixedTicker count={count} /> : <StaleTicker count={count} />}
    </>
  );
}

```

Example. In the box, the button belongs to App, which passes count to a ticker. The **stale** ticker's effect has [], so its interval logs 0 forever while you click. Switch to the **fixed** ticker: its effect lists count, re-runs after every click, and the log follows the button.

Common mistake. Fixing a stale closure by moving the variable out of React (let latest = 0 at module level) or by reading it from the DOM. The component then has two sources of truth. Use the dependency array or an updater function; a useRef mirror is an advanced tool for rare cases.

Cleanup: before the next effect and on unmount

The function an effect returns is its **cleanup**. React runs it before the effect runs again (with the old values) and when the component is removed from the screen (unmount). It undoes the setup: clear the timer, remove the listener, close the connection, abort the request.

```
import { useState, useEffect } from 'react';

function ChatRoom({ roomId }) {
  useEffect(() => {
    console.log(`connect to #${roomId}`);
    return () => console.log(`disconnect from #${roomId}`);
  }, [roomId]);
  return <h2>Welcome to #{roomId}</h2>;
}

function App() {
  const [roomId, setRoomId] = useState('general');
  const [show, setShow] = useState(true);
  return (
    <>
      <select value={roomId} onChange={(e) => setRoomId(e.target.value)}>
        <option value="general">general</option>
        <option value="travel">travel</option>
        <option value="music">music</option>
      </select>
      <button onClick={() => setShow(!show)}>{show ? 'Close the chat' : 'Open the chat'}</button>
      {show && <ChatRoom roomId={roomId} />}
    </>
  );
}
```

Example. In the box, the chat room "connects" in an effect that depends on roomId. Change the room: the console shows the disconnect from the old room **before** the connect to the new one. Then click **Close the chat**: the last cleanup runs.

Common mistake. Writing the cleanup with a new function: window.addEventListener('resize', () => update()) and later window.removeEventListener('resize', () => update()). Two arrow functions are two different objects, so nothing is removed. Keep the handler in a variable inside the effect and pass that same variable to both calls.

StrictMode: why effects run twice in development

<StrictMode> is a development-only checker. On mount it runs every effect, its cleanup and the effect again (setup → cleanup → setup), and it renders each component twice. Production builds do neither.

```
// main.jsx, as created by the Vite React template
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import App from './App.jsx';

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

```
import { StrictMode, useEffect } from 'react';
import { createRoot } from 'react-dom/client';

function App() {
  console.log('render');
  useEffect(() => {
    console.log('connect');
    return () => console.log('disconnect');
  }, []);
  return <p>Connected (look at the console)</p>;
}

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

Example. The box renders its own root wrapped in `<StrictMode>` (that is why it calls `createRoot` itself). Its console shows render twice, then connect, disconnect, connect. Remove the `<StrictMode>` tags and each line appears once.

Common mistake. Adding a `didRun` flag to make an effect run once "because it runs twice". The effect would also run once without a cleanup, so the real bug (no cleanup) survives, and the flag breaks the remount that `StrictMode` is simulating.

You might not need an effect

Effects are for synchronising with **external systems**. To compute a value from props or state, do it during render; to respond to something the user did, do it in the **event handler**. Both are simpler and faster than an effect.

Effect or not?

You want to...	Put it in...
show tasks filtered by the search text	render (a derived const)
send a POST when the user clicks Save	the click / submit handler
clear the form after the POST succeeds	the same handler, after await
load the tasks of the user on screen	an effect with <code>[userId]</code> (or a data library)
start a clock while the component is visible	an effect with a cleanup

```
// Not needed: an effect that reacts to "submitted"
useEffect(() => {
  if (submitted) { postTask(title); setTitle(''); }
}, [submitted]);

// Better: do it where it happens
async function handleSubmit(e) {
  e.preventDefault();
  await postTask(title);
  setTitle('');
}
```

```
import { useState } from 'react';

function App() {
  const [title, setTitle] = useState('');
  const [created, setCreated] = useState([]);

  async function handleSubmit(e) {
    e.preventDefault();
    const res = await fetch('/api/tasks', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ title, done: false, userId: 1 }),
    });
  });
```

```

    if (!res.ok) { console.log('could not create:', res.status); return; }
    const task = await res.json();
    setCreated((prev) => [...prev, task]); // respond to the event here...
    setTitle(''); // ...and reset here, no effect
  }

  return (
    <form onSubmit={handleSubmit}>
      <input value={title} onChange={(e) => setTitle(e.target.value)} placeholder="New task" />
      <button disabled={!title.trim()}>Add</button>
      <p>{created.length} {created.length === 1 ? 'task' : 'tasks'} created</p>
      <ul>{created.map((t) => <li key={t.id}>#{t.id} {t.title}</li>)}</ul>
    </form>
  );
}

```

Example. The box creates tasks against the mock API: the POST and the reset live in the submit handler, and the list title ("2 tasks") is derived. There is no effect in the component at all.

Common mistake. Using an effect to "watch" props and copy them into state: `useEffect(() => setName(user.name), [user])`. The first render shows the old name, a second render fixes it, and any edit the user made is overwritten when user changes. Read `user.name` directly, or reset the form with a key.

Loading data

Fetching data in an effect

To load data for what is on the screen, start the request in an effect whose dependencies are the inputs of the request (`[]`, `[id]`, `[query]`), check `res.ok`, and store the result in state. The effect function itself cannot be `async`: define an `async` function inside it and call it.

```

useEffect(() => {
  let ignore = false;
  async function load() {
    try {
      const res = await fetch(`/api/tasks?userId=${userId}`);
      if (!res.ok) throw new Error(`HTTP ${res.status}`);
      const data = await res.json();
      if (!ignore) setTasks(data);
    } catch (err) {
      if (!ignore) setError(err.message);
    }
  }
  load();
  return () => { ignore = true; }; // a newer run or an unmount wins
}, [userId]);

```

```

import { useState, useEffect } from 'react';

function App() {
  const [userId, setUserId] = useState(1);
  const [tasks, setTasks] = useState([]);
  const [error, setError] = useState(null);

  useEffect(() => {
    let ignore = false;
    async function load() {
      try {
        const res = await fetch(`/api/tasks?userId=${userId}`);
        if (!res.ok) throw new Error(`HTTP ${res.status}`);
        const data = await res.json();
        if (!ignore) { setTasks(data); setError(null); }
      } catch (err) {
        if (!ignore) setError(err.message);
      }
    }
  }, [userId]);
}

```

```

    }
    load();
    return () => { ignore = true; };
  }, [userId]);

  return (
    <>
      <select value={userId} onChange={(e) => setUserId(Number(e.target.value))}>
        <option value={1}>User 1</option>
        <option value={2}>User 2</option>
        <option value={3}>User 3</option>
      </select>
      {error && <p role="alert">Could not load: {error}</p>}
      <ul>{tasks.map((t) => <li key={t.id}>{t.title}</li>)}</ul>
    </>
  );
}

```

Example. The box loads the tasks of a user from the mock API (it answers `/api/...` after 300 ms and logs each request). Change the user: the effect re-runs with the new id, the cleanup of the previous run flips its `ignore` flag, and the list switches over.

Common mistake. Fetching in the component body: `fetch(url).then((r) => r.json()).then(setTasks)`. Every render starts a request, every response sets state, every state change renders again: an endless loop of requests. Fetches that load data for the screen go in an effect; fetches caused by a click go in the handler.

Loading, error, empty and success

A view that loads data has **four states**, and the user must be able to tell them apart: **loading** (the request is on its way), **error** (it failed, with a way to retry), **empty** (it worked and there is nothing to show) and **success** (the data).

The four states of a list view

State	Condition	Show
loading	<code>status === 'loading'</code>	"Loading tasks..." (or a skeleton)
error	<code>status === 'error'</code>	the message + a Try again button
empty	<code>status === 'success' && tasks.length === 0</code>	"No tasks match ..." + what to do next
success	<code>status === 'success'</code>	the list

```

import { useState, useEffect } from 'react';

function App() {
  const [search, setSearch] = useState('');
  const [broken, setBroken] = useState(false);
  const [attempt, setAttempt] = useState(0);
  const [status, setStatus] = useState('loading');
  const [tasks, setTasks] = useState([]);
  const [error, setError] = useState('');

  useEffect(() => {
    let ignore = false;
    setStatus('loading');
    fetch(`/api/${broken ? 'taskz' : 'tasks'}?search=${encodeURIComponent(search)}`)
      .then((res) => { if (!res.ok) throw new Error(`The server answered ${res.status}`); return res.json(); })
      .then((data) => { if (!ignore) { setTasks(data); setStatus('success'); } })
      .catch((err) => { if (!ignore) { setError(err.message); setStatus('error'); } });
    return () => { ignore = true; };
  }, [search, broken, attempt]);

  return (
    <>
      <input placeholder="Search tasks" value={search} onChange={(e) => setSearch(e.target.value)} />
      <label><input type="checkbox" checked={broken} onChange={(e) => setBroken(e.target.checked)} /> Break the
    URL</label>
    <div aria-live="polite">

```

```

    {status === 'loading' && <p>Loading tasks...</p>}
    {status === 'error' && (
      <p role="alert">{error}. <button onClick={() => setAttempt(attempt + 1)}>Try again</button></p>
    )}
    {status === 'success' && tasks.length === 0 && <p>No tasks match "{search}"</p>}
    {status === 'success' && tasks.length > 0 && (
      <ul>{tasks.map((t) => <li key={t.id}>{t.title}</li>)}</ul>
    )}
  </div>
</>
);
}

```

Example. The box searches the mock API with an 800 ms delay so you can see each state. Type xyz for the empty state; tick **Break the URL** for the error state (the API answers 404) and then use **Try again** after unticking it.

Common mistake. Starting with loading = false and tasks = []. The very first render then shows the empty state ("No tasks yet") for a moment before the request has even started, which looks like a real answer. Start in the loading state: useState('loading').

Cancelling a fetch with AbortController

An **AbortController** **cancels a request: pass controller.signal to fetch, and call controller.abort() in the effect's cleanup. The browser drops the request and the promise rejects with an `AbortError`, which you ignore.**

```

useEffect(() => {
  const controller = new AbortController();
  fetch(`/api/tasks?search=${query}`, { signal: controller.signal })
    .then((res) => { if (!res.ok) throw new Error(`HTTP ${res.status}`); return res.json(); })
    .then(setTasks)
    .catch((err) => {
      if (err.name === 'AbortError') return; // we cancelled it: not an error
      setError(err.message);
    });
  return () => controller.abort(); // outdated: cancel it
}, [query]);

```

```

import { useState, useEffect } from 'react';

function App() {
  const [query, setQuery] = useState('');
  const [tasks, setTasks] = useState([]);

  useEffect(() => {
    const controller = new AbortController();
    fetch(`/api/tasks?search=${encodeURIComponent(query)}`, { signal: controller.signal })
      .then((res) => { if (!res.ok) throw new Error(`HTTP ${res.status}`); return res.json(); })
      .then((data) => {
        console.log(`results for "${query}": ${data.length}`);
        setTasks(data);
      })
      .catch((err) => {
        if (err.name === 'AbortError') console.log(`${query} aborted`);
        else console.log('failed:', err.message);
      });
    return () => controller.abort();
  }, [query]);

  return (
    <>
      <input placeholder="Type quickly" value={query} onChange={(e) => setQuery(e.target.value)} />
      <ul>{tasks.map((t) => <li key={t.id}>{t.title}</li>)}</ul>
    </>
  );
}

```

Example. The box searches with an 800 ms delay and aborts in the cleanup. Type three letters quickly: the console shows the first two requests cancelled and only the last one answered. The visualiser has the same situation step by step, and the version without a cleanup where the old response wins.

Common mistake. Treating the `AbortError` as a failure: the catch sets `status = 'error'`, and every keystroke flashes "Something went wrong". Check `err.name === 'AbortError'` first and do nothing in that case.

Hooks: rules and reuse

The rules of hooks (and why)

Call hooks (`useState`, `useEffect`, **any** `use...` **function**) **only at the top level** of a component or a custom hook: **never inside conditions, loops, nested functions or after an early return**. React identifies each hook by its **call order**, which must be the same on every render.

```
// Wrong: the hook only runs sometimes
function TaskDetail({ task }) {
  if (!task) return <p>Select a task</p>;
  const [editing, setEditing] = useState(false); // slot depends on task
  ...
}

// Right: every hook first, then the early return
function TaskDetail({ task }) {
  const [editing, setEditing] = useState(false);
  if (!task) return <p>Select a task</p>;
  ...
}
```

```
import { useState } from 'react';

function App() {
  const [details, setDetails] = useState(false);

  if (details) {
    // Breaks the rules: this hook only exists on some renders
    const [note, setNote] = useState('a note');
    console.log(note);
  }

  return (
    <label>
      <input type="checkbox" checked={details} onChange={(e) => setDetails(e.target.checked)} />
      Show details (this breaks the component)
    </label>
  );
}
```

Example. The box breaks the rule on purpose: an extra `useState` runs only when the checkbox is ticked. The first render works; tick the box and React throws, because the second render calls one more hook than the first. Move the hook out of the `if` to fix it.

Common mistake. Calling a hook inside an event handler: `onClick={() => { const [x, setX] = useState(0); ... }}`. Handlers run outside rendering, so React cannot attach the state to the component and throws "Invalid hook call". Declare the state at the top level and use its setter in the handler.

Custom hooks: reusing stateful logic

A **custom hook** is a function whose name starts with `use` and that calls other hooks. It packages a piece of stateful logic (state + effects) so several components can reuse it. Each component that calls it gets its own independent state.

```
function useToggle(initial = false) {
  const [on, setOn] = useState(initial);
  const toggle = () => setOn((v) => !v);
  return [on, toggle];
}

// in any component
const [open, toggleOpen] = useToggle();
```

```
import { useState } from 'react';

function useToggle(initial = false) {
  const [on, setOn] = useState(initial);
  const toggle = () => setOn((v) => !v);
  return [on, toggle];
}

function Panel({ title, children }) {
  const [open, toggleOpen] = useToggle();
  return (
    <section>
      <button aria-expanded={open} onClick={toggleOpen}>{open ? '▼' : '▶'} {title}</button>
      {open && <p>{children}</p>}
    </section>
  );
}

function App() {
  return (
    <>
      <Panel title="What is state?">Data a component remembers between renders.</Panel>
      <Panel title="What is an effect?">Code that synchronises with something outside React.</Panel>
    </>
  );
}
```

Example. The box defines `useToggle` and uses it in two `Panel` components. Opening one does not open the other: each call has its own state.

Common mistake. Expecting two components that call the same custom hook to see the same data, for example a `useCart()` in the header and in the product page. Each call has its own state, so adding to one cart does not change the other. Shared data needs one owner: a common parent or a context provider.

A custom hook for loading data: `useFetch`

`useFetch(url)` **wraps the whole loading pattern (status, data, error, abort, retry) and returns** `{ data, loading, error, reload }`. **Every view that loads something becomes a few lines, and the pattern is fixed in one place.**

```
import { useState, useEffect } from 'react';

function useFetch(url) {
  const [state, setState] = useState({ data: null, loading: true, error: null });
  const [attempt, setAttempt] = useState(0);

  useEffect(() => {
    const controller = new AbortController();
    setState((s) => ({ ...s, loading: true, error: null }));
    fetch(url, { signal: controller.signal })
      .then((res) => { if (!res.ok) throw new Error(`HTTP ${res.status}`); return res.json(); })
      .then((data) => setState({ data, loading: false, error: null }));
  }, [url, attempt]);
}
```

```

    .catch((err) => { if (err.name !== 'AbortError') setState({ data: null, loading: false, error: err.message
}); });
    return () => controller.abort();
  }, [url, attempt]);

  return { ...state, reload: () => setAttempt((a) => a + 1) };
}

function TaskCount() {
  const { data, loading } = useFetch('/api/tasks?done=false');
  return <p>{loading ? '...' : `${data.length} pending tasks`}</p>;
}

function TaskList() {
  const { data, loading, error, reload } = useFetch('/api/tasks');
  if (loading) return <p>Loading...</p>;
  if (error) return <p role="alert">{error} <button onClick={reload}>Try again</button></p>;
  if (data.length === 0) return <p>No tasks yet.</p>;
  return (<><button onClick={reload}>Reload</button><ul>{data.map((t) => <li key={t.id}>{t.title}</li>)}</ul></>);
}

function App() {
  return (<><TaskCount /><TaskList /></>);
}

```

Example. The box defines `useFetch` and uses it in two components: a task count and a task list, each with its own request (watch the two `[network]` lines). **Reload** in the list calls `reload()`, which re-runs only that component's effect.

Common mistake. Passing a URL built from an object or array that is recreated on every render, or calling `useFetch` with a new options object each time and listing it as a dependency: the effect then re-runs after every render, which means an endless request loop. Keep the dependencies primitive (a URL string, an id).

A custom hook for persistence: `useLocalStorage`

`useLocalStorage(key, initial)` **works like** `useState`, **but reads its first value from** `localStorage` **and writes every change back, so the value survives a page reload. Reading happens once, in a lazy initialiser; writing is an effect.**

```

function useLocalStorage(key, initial) {
  const [value, setValue] = useState(() => {
    try {
      const saved = localStorage.getItem(key);
      return saved === null ? initial : JSON.parse(saved);
    } catch {
      return initial; // blocked or invalid: use the default
    }
  });

  useEffect(() => {
    try { localStorage.setItem(key, JSON.stringify(value)); } catch { /* full or blocked */ }
  }, [key, value]);

  return [value, setValue];
}

// usage: exactly like useState
const [theme, setTheme] = useLocalStorage('theme', 'light');

```

```

import { useState, useEffect } from 'react';

function useLocalStorage(key, initial) {
  const [value, setValue] = useState(() => {
    try {
      const saved = localStorage.getItem(key);
      return saved === null ? initial : JSON.parse(saved);
    } catch {
      console.log(`storage blocked: "${key}" starts from its default`);
    }
  });

```

```
    return initial;
  }
});

useEffect(() => {
  try { localStorage.setItem(key, JSON.stringify(value)); }
  catch { console.log(`storage blocked: "${key}" kept in memory only`); }
}, [key, value]);

return [value, setValue];
}

function App() {
  const [theme, setTheme] = useLocalStorage('theme', 'light');
  const [draft, setDraft] = useLocalStorage('draft', '');
  return (
    <div style={{ padding: 8, background: theme === 'dark' ? '#222' : '#fff', color: theme === 'dark' ? '#eee' : '#111' }}>
      <button onClick={() => setTheme(theme === 'dark' ? 'light' : 'dark')}>Theme: {theme}</button>
      <p><textarea value={draft} onChange={(e) => setDraft(e.target.value)} placeholder="Draft note" /></p>
    </div>
  );
}
```

Example. The box uses the hook for a theme choice and a draft note. This preview runs in a locked-down frame where storage is blocked, so the `catch` branches run (the console says so) and the values reset when the preview reloads; in your own app they survive a page reload. That is exactly why the `try/catch` is there.

Common mistake. Reading storage in the body without the lazy initialiser: `useState(JSON.parse(localStorage.getItem('tasks')))`. It parses on every render (the result is ignored after the first), and the first time it runs `JSON.parse(null)`, which returns `null` instead of your default, so `tasks.map` crashes.